

Project Report

Project Name: PAWSE

Reference ID: AHBH6CIB

1. Executive Summary

PAWSE is a desktop productivity application that transforms the traditional Pomodoro technique into a relaxing and enjoyable experience through interactive virtual cats. By offering a low-stimulation "Cat Mode" that communicates work/break state through animation rather than just a timer, PAWSE directly targets the two most common reasons Pomodoro techniques fail: jarring alarms and timer anxiety. The result is a stress-free, offline-first productivity companion that makes focus sessions feel like spending time with a pet rather than watching a clock.

2. Project Overview

2a. Why you're building what you're building

Traditional Pomodoro timers rely on two design choices that quietly work against the people using them: a jarring alarm at the end of a session and a constantly visible numeric countdown. Research shows that sudden, alarm-like sounds do not just startle the listener but disrupt attention and short-term memory, producing more mistakes and slower reaction times on cognitive tasks compared to quieter or continuous sounds (Monteiro et al., 2018). In other words, the very sound meant to help someone "snap back" into focus after a break can instead result in a disrupted concentration. The visible countdown creates a second, related problem: constantly seeing the numbers pulls attention away from the task itself, since part of the brain is busy tracking the clock instead of focusing on the work (Zakay & Block, 2004). This creates a sense of time pressure, which research has linked to reduced focus and weaker task performance (Sussman et al., 2022). So ironically, the tool that's meant to help you focus ends up being one more thing distracting you.

PAWSE is designed to solve both problems directly rather than treat them as unavoidable side effects of using a timer. Instead of one fixed alarm, the app offers a small library of individually adjustable sounds — ambient background audio, a continuous purring loop for work time, a gentle cat-meow transition cue instead of a jarring buzzer, a soft ticking option, and reactive meow sounds during break-time interactions — so a user who finds any single sound disruptive can turn it off or swap it out in settings instead of being stuck with a one-size-fits-all alarm. To address the countdown problem, PAWSE's Cat Mode replaces the visible numeric timer with ambient cat animation, letting users sense whether it's work time or break time without the attention cost of actively watching a countdown tick down. Looking ahead, we see room to take the sound side of this even further: a future version of PAWSE could incorporate deeper research into which specific sounds or music genres are empirically shown to support productivity and sustained concentration, rather than relying only on gentle, non-jarring audio by default.

2b. How it relates to the theme

PAWSE embraces the **World Cat Domination Day** theme by transforming cats from simple visual mascots into the heart of the productivity experience. Instead of treating cats as decorative elements, each companion has a distinct personality inspired by common cat stereotypes and directly influences the user's focus routine through its unique Pomodoro schedule.

Beyond productivity, PAWSE encourages appreciation for cats by creating positive interactions throughout the day. During breaks, users can pet their virtual companion to trigger unique animations, playful meows, and fun random cat facts that celebrate feline behavior and educate users about cats in an engaging way.

The application also promotes a calmer and more enjoyable working environment through comforting cat-inspired experiences such as ambient purring, expressive animations, and gentle meow sounds. By making cats a source of relaxation, learning, and motivation, PAWSE contributes to the hackathon's vision of helping cats "take over the world" in a wholesome

way—bringing more feline charm into everyday life while encouraging healthier productivity habits.

2c. Target Audience

Our target audience is students and professionals who use timers during their productivity time and who are also cat lovers looking for a fun, stress-free alternative to traditional timer apps. It's especially suited to people who've tried standard Pomodoro apps before but found the alarms jarring or the countdown itself distracting or anxiety-inducing.

3. Key Features

☒ **Themed Timers**

Choose from three unique cat companions, each inspired by a familiar cat personality and paired with a productivity style:

- **Tux (Tuxedo Cat):** 25-minute work, 5-minute short break, 20-minute long break — a balanced companion for steady productivity.
- **Ginger (Orange Cat):** 15-minute work, 3-minute short break, 12-minute long break — energetic and playful, ideal for users who prefer frequent breaks.
- **Void (Black Cat):** 50-minute work, 10-minute short break, 40-minute long break — calm and mysterious, designed for deep focus sessions.

☒ **Dual Focus Modes**

PAWSE offers two minimalist viewing modes to accommodate different work styles:

- **Timer Mode** displays only the countdown timer for users who prefer a clean, distraction-free workspace.
- **Cat Mode** shows only the cat and its animation, communicating work/break state without a visible countdown, for users prone to timer anxiety.

Both modes reveal sound, play/pause, and skip controls on hover.

☒ **Calming Sound Experience**

Instead of harsh buzzer alarms, PAWSE creates a relaxing atmosphere using continuous ambient music and purring sounds during work sessions. When it's time for a break, the purring fades and is replaced by a gentle meow, providing a softer and less disruptive transition between sessions.

☑ **Interactive Break-Time Companion**

Cat interactions become available only during break sessions, encouraging users to actually step away from work. Clicking or petting the selected cat triggers a unique animation, an adorable meow, and a randomly generated cat fact displayed in a speech bubble, making breaks more rewarding and enjoyable.

☑ **Dashboard**

A dedicated dashboard shows the user's total focus time for the day, the number of Pomodoros completed, and their favorite cat companion, alongside a weekly bar graph ("Weekly Flow") visualizing daily focus activity. Users can navigate back through the current week plus the previous three weeks to review their recent progress.

4. Technology Stack

Layer	Technology
Frontend	HTML5, CSS3, JavaScript
Framework	Electron
Backend	Node.js
Database	SQLite3
Package Manager	npm
Version Control	Git, GitHub

5. Technical Architecture

PAWSE adheres to a strict **Model-View-Controller (MVC)** directory structure, separating the Node.js backend (src/main/) from the Chromium frontend (src/renderer/).

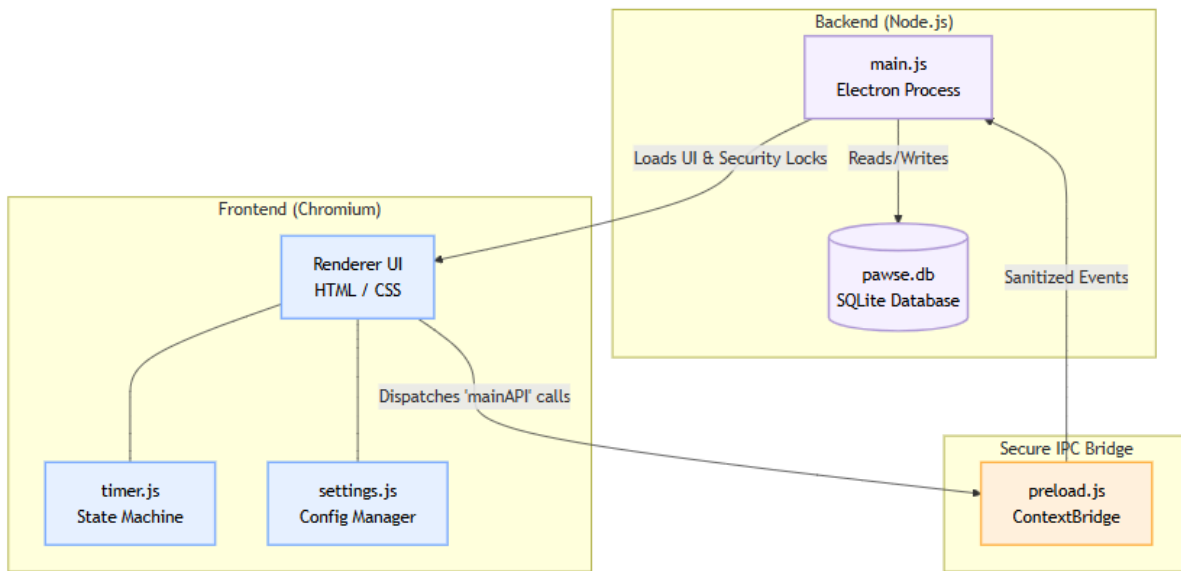


Figure 1. System Architecture Diagram

System Components

- **Electron Main Process (src/main/main.js):** Instantiates the BrowserWindow, completely locking down native OS integration, removing window frames, and handling lifecycle hooks.
- **Renderer Process (src/renderer/):** Houses the index.html, settings.html, and timer/timer.html views. Styles are universally governed by a centralized index.css root design token system.
- **Preload Bridge (src/main/preload.js):** Acts as the strict IPC gateway. Instead of blindly exposing ipcRenderer, it exclusively exposes a highly limited mainAPI object for specific tasks (like savesession or minimize).
- **Database Engine (src/main/database.js):** Handles the raw SQLite3 connection logic. Intercepts session completion events to execute parameterized INSERT statements tracking deep analytics per companion

6. Testing Matrix

Feature / Flow	Steps	Expected Result	Actual Result	Pass / Fail
Select Cat	Choose Tux, Ginger, or Void	Correct timer durations are loaded	Timer changes according to selected cat	Pass
Start/Pause Timer	Press Play then Pause	Timer starts and pauses correctly	Timer behaves as expected	Pass
Skip Session	Press Skip	Current session immediately advances	Next session loads correctly	Pass
Work Session Audio	Start work session	Purring audio plays continuously	Purring plays correctly	Pass
Break Transition	Wait for work session to finish	Gentle meow plays and purring stops	Audio transitions correctly	Pass
Cat Interaction	Click the cat during break	Animation, meow, and random cat fact appear	All interactions trigger successfully	Pass
Disable Interaction During Work	Click cat while working	Cat should not respond	Interaction is disabled	Pass
Timer Mode	Switch to Timer Mode	Only timer is displayed	Mode works correctly	Pass
Cat Mode	Switch to Cat Mode	Only animated cat is displayed	Mode works correctly	Pass
Sound Toggle	Toggle sound on/off	Audio enables and disables properly	Feature functions correctly	Pass

7. Security Scan Reports (Aikido)

The screenshot shows the Aikido web interface. On the left is a dark sidebar with navigation links: Dashboard, Feed, AutoFix, Assets (Repositories: 1, Containers: 0, Clouds: 0, Virtual Machines: 0, Domains: 1), Attack (Pentests, Code Audit), Protect (Devices: 0, Firewall: 0), and More (Integrations, Code Quality). The main content area is titled 'Repositories' and shows '1 active repos'. A 'Start Scan' button is in the top right. Below the title, there are tabs for 'Repositories', 'Pull Requests', and 'Checks'. A search bar and a 'Manage Repos' button are also present. A table lists the repository 'Pawse' with the following data:

Repo name	Language	Issues	Ignored	Last scan
Pawse	JS	0 0 0 0	3	3m ago

Figure 2. Repository Scan Results

The screenshot shows the Aikido web interface for the 'Pawse' repository. The sidebar is the same as in Figure 2. The main content area is titled 'Pawse' and shows '0 issues'. There are buttons for 'Scan Branch' and 'Start Scan'. Below the title, there are tabs for 'Issues', 'Settings', and 'Checks'. A search bar and a '1 issue solved just now' notification are present. A table lists the repository 'Pawse' with the following data:

Type	Name	Severity	Location	Fix time	Status
No Issues found.					

A notification box at the bottom right says: 'Scan complete. Your scans have completed. You can now refresh the queue to view all new results.' with a 'View feed' link and a close button.

Figure 3. PAWSE Scan Results

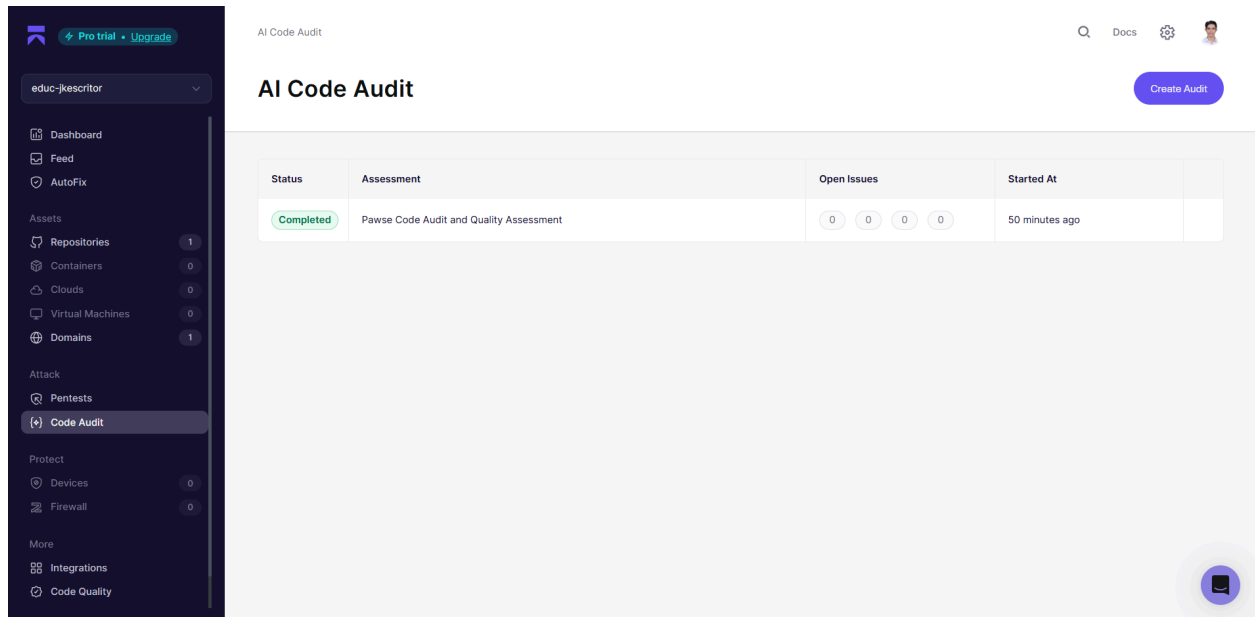


Figure 4. AI Code Audit Results

8. Future Improvements

☐ Personalized Companion

Allow users to create their own cat companion by customizing its appearance, choosing work and break durations, and giving it a personalized name. This would let users tailor both the visual experience and the Pomodoro schedule to suit their productivity preferences.

☐ Interactive Cat Environments

Expand each cat's space into a themed room that reflects its personality. Each companion would have unique furniture, toys, decorations, and resting spots that make the desktop companion feel more alive and immersive.

☐ Expanded Break-Time Interactions

Introduce additional interactions during breaks, such as feeding, playing with toys, giving treats, or watching the cat explore its environment. Different interactions could trigger exclusive animations, sounds, and dialogue to make breaks feel more rewarding.

☐ Achievement & Progress System

Implement a progression system where users earn achievements, unlock cosmetics, or collect items for their personalized cat by completing focus sessions. This would encourage consistency and make productivity feel more rewarding over time.

☐ **Enhanced Audio & Accessibility**

Provide a wider selection of ambient sounds, meows, and transition effects, while also allowing users to customize sound volumes, animation intensity, and accessibility settings to create a more personalized experience.

☐ **Expanded Cat Knowledge Base**

Increase the collection of cat facts with topics such as breeds, health, behavior, history, and fun trivia. This keeps interactions fresh while helping users learn something new during every break.

9. Tools You Used

- ☒ **Figma** - Designed the application's user interface and prototype. Dev Mode was used to translate layouts into CSS.
- ☒ **Visual Studio Code (VS Code)** – Primary development environment.
- ☒ **Electron** – Desktop application framework.
- ☒ **Node.js & npm** – Runtime environment and dependency management.
- ☒ **Git & GitHub** – Version control and collaborative development.
- ☒ **Antigravity CLI** – AI-assisted development and code generation.
- ☒ **Aikido** – Security scanning to catch vulnerabilities and dependency issues before submission.
- ☒ **Google Docs** – Project report documentation.
- ☒ **OBS Studio** – Screen recording for the project demonstration.
- ☒ **Adobe Premiere Pro** – Editing and polishing the demo video.

11. Learnings & Takeaways

Building PAWSE was a rigorous but deeply rewarding journey that tested our limits across system architecture, user experience, and team collaboration. Throughout the development lifecycle, we gained several key takeaways:

1. Empathy-Driven Design

- We learned the value of stepping out of the "developer mindset" and adopting the perspective of the target user. By actively simulating and testing with real users how users experience burnout and time anxiety, we made critical decisions about which features to include—and more importantly, which ones to exclude—to maintain the user's focus. This empathy directly inspired our unique "Cat Mode" design, proving that what a system hides is just as important as what it shows. Ultimately, we realized that productivity tools should improve not only efficiency but also the user's emotional experience while working.

2. Problem Solving

- We successfully identified key friction points in traditional Pomodoro timers, such as harsh alarms, rigid structures, and lack of motivation. We learned how to alleviate these pain points through thoughtful programming and design while staying true to our cat theme. Balancing serious productivity mechanics with playful companion dynamics taught us how to create software that is both highly functional and emotionally engaging, while maintaining a strong balance between creativity and usability.

3. UI/UX in Productivity Apps

- This project expanded our understanding of UI/UX beyond visual design alone. We learned how animations, ambient audio, gentle sound effects, and interactive feedback work together to create a more engaging productivity experience. We also strengthened our understanding of UI/UX principles specific to productivity applications and gained insight into the design differences between desktop applications and other interfaces. We

discovered that a truly premium application orchestrates visuals, sound, and movement into one cohesive user experience.

4. Technical Execution & DevSecOps

- Building a desktop application significantly strengthened our technical skills. We gained practical knowledge in Electron desktop development, system architecture, state management for timed events, local durable execution (fault tolerance), audio and animation integration, and secure inter-process communication (IPC) between the Chromium frontend and Node.js backend using contextBridge protocols. We also applied DevSecOps principles by hardening the application through OS-level sandboxing, Content Security Policies (CSP), and protection against injection vulnerabilities.

5. Agile Iteration

- We quickly learned that rigid planning often falls short when technical issues arise. By embracing an Agile development approach, we continuously iterated on our ideas, refactored our architecture when necessary, and removed features that did not meet our quality standards. This adaptability allowed us to maintain a polished, technically sound final product despite evolving project requirements.

6. Professional Team Dynamics & Transparent Communication

- Beyond technical development, we learned how to function as a professional engineering team. We delegated tasks effectively, collaborated efficiently using Git and GitHub, maintained high quality standards, and communicated openly throughout the project. By treating the project as though we were developing for a professional client, we ensured that every deliverable aligned with our shared vision and was completed successfully.

12. Acknowledgments

We'd like to thank the #hackthekitty organizers for choosing a theme that let us combine two things we genuinely love: cats and app development, which is directly related to what we're

pursuing right now. The theme made programming feel fun and a little more cute along the way, but it also pushed us to think beyond the usual apps we use every day—to actually build on top of them and fix real problems we've encountered, backed by reliable studies rather than just guesswork. We're also grateful for the tools and resources the organizers provided, like Temporal, Aikido, and Kiro. These weren't tools we were familiar with going in, but getting to explore and actually apply them in a real project means we now understand them well enough to likely use them again in future work.

We'd also like to credit the open-source projects, libraries, and communities that made PAWSE possible—particularly the Electron and Node.js ecosystems, whose documentation and tooling made our main/renderer architecture and offline-first design achievable within the hackathon timeframe.

And lastly, we want to thank cats! Especially our own cat, Nabi, who was the inspiration behind PAWSE's logo.

13. References

- Monteiro, R., Tomé, D., Neves, P., Silva, D., & Rodrigues, M. A. (2018). The interactive effect of occupational noise on attention and short-term memory: A pilot study. *Noise & Health*, 20(96), 190–198. https://doi.org/10.4103/nah.NAH_3_18
- Sussman, A. B., Sharma, E., & Alter, A. L. (2022). Feeling rushed? Perceived time pressure and cognitive function. *Journal of Experimental Psychology: General*, 151(9), 2087–2100. <https://doi.org/10.1037/xge0001173>
- Zakay, D., & Block, R. A. (2004). The role of attention in time estimation. *Cognitive Brain Research*, 21(2), 183–204. <https://doi.org/10.1016/j.cogbrainres.2004.02.011>

Submission Checklist

- ☒ Video demo (HD or at least 720p)
- ☒ README.md (prerequisites, run instructions, configuration)
- ☒ Project report (this document, in documentation/)
- ☒ Source code in src/
- ☒ No unrelated files, executables, auto-generated code, or package folders (e.g. node_modules/, build/, dist/, vendor/)